

Capítulo 11

Mutación¹

11.1 Memoria para funciones

Sin importar qué tan seguido usemos una función con el mismo argumento, siempre obtendremos el mismo resultado. Las funciones simplemente no tienen memoria sobre sus usos anteriores.

Sin embargo, muchos programas deben recordar algo sobre sus aplicaciones anteriores. Recuerde que un programa típicamente consiste en muchas funciones. En el pasado siempre hemos supuesto que existe una función principal, y que todas las otras funciones auxiliares son invisibles al usuario. En algunos casos, un usuario puede esperar más de un servicio o funcionalidad a partir de un programa, y cada servicio es mejor implementarlo como una función. Cuando un programa provee más de una función como un servicio al usuario, es común que, por conveniencia o porque agregamos alguna interfaz de usuario, las funciones deban tener memoria.

Como este punto es difícil de comprender, estudiaremos un ejemplo sencillo: manejar números de teléfono en una agenda o libreta de direcciones. Un programa de libreta de direcciones provee usualmente al menos dos servicios:

1. un servicio para buscar el número de teléfono de cierta persona; y
2. un servicio para agregar un nombre y un número de teléfono a la libreta.

Estos dos servicios corresponden a dos funciones. Además, introducimos una definición de una estructura para mantener la asociación nombre-número:

```
1 import estructura
2 from lista import *
3
4 #registro: nombre(str) numero(int)
5 estructura.crear("registro", "nombre numero")
6 libretaDeDirecciones = crearLista(registro("Maria", 1), \
7                                   crearLista(registro("Pedro", 2), listaVacía))
8
9 # buscar: lista(registro) str -> num or False
10 # busca nombre en libreta y devuelve el numero correspondiente
11 # si no encuentra nombre, retorna False
```

¹Parte de este capítulo fue traducido al español y adaptado de: M. Felleisen et al.: *How to Design Programs*, MIT Press. Disponible en: www.htdp.org

```

12 def buscar(libreta, nombre):
13     ...
14
15 # agregarALibreta: str num -> None
16 # agrega nombre y numero a libretaDeDirecciones
17 def agregarALibreta(nombre, numero):
18     ...

```

La primera función recibe como parámetro una lista de asociaciones nombre-número (registro), llamada `libretaDeDirecciones`, y un nombre. La función produce un número, si el nombre está en la lista, o `False` en caso contrario. La segunda función es radicalmente distinta de lo que hemos visto hasta el momento. El usuario debe invocarla con un nombre y un número; cualquier búsqueda posterior (invocación a `buscar`) con ese nombre debería producir ese número.

Imaginemos una interacción con nuestra libreta:

```

1 >>> buscar(libretaDeDirecciones, "Maria")
2 1
3 >>> buscar(libretaDeDirecciones, "Claudio")
4 False
5 >>> agregarALibreta("Claudio", 5)
6 >>> buscar(libretaDeDirecciones, "Claudio")
7 5

```

Las dos primeras instrucciones confirman que María tiene el número 1 y que no hay un número registrado para Claudio. La tercera instrucción agrega el número 5 para Claudio. La última instrucción muestra que el mismo uso de nuestra función `buscar` ahora produce el número de teléfono esperado.

En el pasado, la única forma de lograr el mismo efecto es que el usuario hubiera editado en el código de nuestro programa la definición de `libretaDeDirecciones`. Sin embargo, no queremos que los usuarios tengan que editar nuestros programas para obtener el resultado deseado. De hecho, ellos no *deberían* tener acceso al código de nuestros programas. Por lo tanto, estamos forzados a proveer una interfaz con una función que permita tales cambios.

Veamos la definición de nuestra libreta de direcciones en el siguiente código. La función `agregarALibreta` consume un nombre (string) y un número de teléfono (entero). Su cuerpo lo que hace es una asignación de la misma variable agregando el registro correspondiente. La función `buscar` consume una libreta de direcciones y un nombre; su resultado es el número correspondiente o `False` si el nombre no está en la libreta.

```

1 import estructura
2 from lista import *
3
4 #registro: nombre(str) numero(int)
5 estructura.crear("registro", "nombre numero")
6 libretaDeDirecciones = listaVacía
7
8 # agregarALibreta: str num -> None
9 # agrega nombre y numero a libretaDeDirecciones
10 def agregarALibreta(nombre, numero):
11     global libretaDeDirecciones
12     libretaDeDirecciones = crearLista(registro(nombre, numero), \
13                                     libretaDeDirecciones)

```

```
14
15 # buscar: lista(registro) str -> num or False
16 # busca nombre en libreta y devuelve el numero correspondiente
17 # si no encuentra nombre, retorna False
18 def buscar(libreta, nombre):
19     if vacia(libreta):
20         return False
21     elif cabeza(libreta).nombre == nombre:
22         return cabeza(libreta).numero
23     else:
24         return buscar(cola(libreta), nombre)
```

11.2 Diseñar funciones con memoria

La sección anterior motivó la idea de funciones con memoria. En esta sección discutiremos el diseño de funciones con memoria.

Diseñar funciones con memoria requiere tres pasos importantes:

1. Debemos determinar que un programa requiere memoria.
2. Debemos identificar los datos que irán en la memoria.
3. Debemos entender cuál o cuáles servicios están destinados a modificar la memoria, y cuál o cuáles van a usar la memoria.

La necesidad por el primer paso es obvia. Una vez que sabemos que un programa requiere memoria, debemos realizar un análisis de los datos para la memoria de éste. Es decir, debemos determinar qué clase de datos se pondrán y se sacarán de ahí. Finalmente, debemos diseñar cuidadosamente las funciones para el programa que van a cambiar la memoria.

11.2.1 La necesidad de tener memoria

Afortunadamente, es relativamente fácil darse cuenta de cuándo un programa necesita mantener memoria. Como se discutió anteriormente, hay dos situaciones. La primera involucra programas que proveen más de un servicio a los usuarios. Cada servicio corresponde a una función. Un usuario puede aplicar esas funciones en el intérprete de Python, o pueden ser aplicadas en respuesta a la acción de un usuario en una interfaz gráfica. La segunda involucra un programa que provee un solo servicio y que es implementado con una sola función a nivel de usuario. Pero el programa puede producir diferentes resultados cuando es aplicada con los mismos argumentos o parámetros.

Volvamos al ejemplo de la libreta de direcciones. Vimos cómo una función agrega registros a la libreta de direcciones y otra busca nombres en ella. Claramente, el uso del “servicio de adición” afecta los usos posteriores del “servicio de búsqueda”, y por lo tanto requiere memoria. De hecho, la memoria en este caso corresponde a un objeto físico natural: la libreta de direcciones que la gente guardaba antes de la existencia de dispositivos electrónicos.

La segunda clase de memoria también tiene ejemplos clásicos. Uno de ellos es el típico contador. Cada vez que se aplica la función producirá un número distinto:

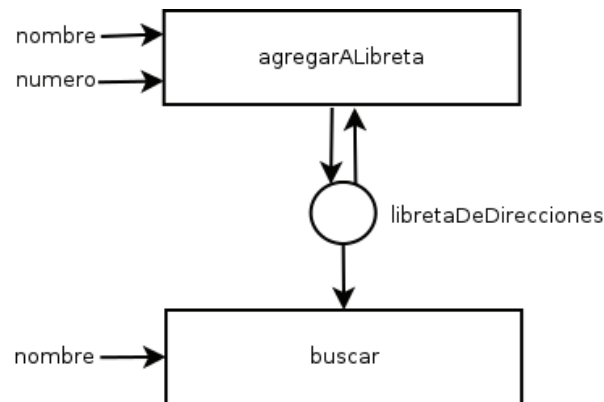


Figura 11.1: Organización de programas con memoria, para el ejemplo de la libreta de direcciones.

```

1 contador = 0
2 # cuenta: None -> int
3 # inicialmente devuelve 0; en la aplicacion numero i devuelve i
4 def cuenta():
5     global contador
6     contador = contador + 1
7     return contador
  
```

Cada vez que se aplique la función, devolverá un valor distinto. En este caso la mutación o modificación de la memoria se hace al modificar la variable `contador`. En Python es necesario usar la instrucción `global` antes de la variable que se quiere modificar. Si no se incluye la instrucción, el intérprete supondrá que se está definiendo una nueva variable. El ejemplo de arriba no funcionaría sin esta instrucción, dado que `contador` aún no ha sido definida.

En general, cuando se analiza el enunciado de un problema, es conveniente dibujar la organización del programa con memoria, como en la Figura 11.1. Cada servicio es representado como una caja rectangular, las flechas que van hacia las cajas indican qué tipo de dato consume el servicio; las flechas que salen indican lo que producen. La memoria se representa con círculos. Una flecha de un círculo hacia una caja significa que el servicio usa la memoria como una entrada; una flecha hacia un círculo significa que el servicio cambia la memoria. La figura muestra que los servicios comúnmente usan la memoria y la modifican.

11.2.2 Memoria y variables de estado

La memoria se implementa con definiciones de variable. En principio, una sola variable es suficiente para implementar todas las necesidades de memoria, como hemos visto hasta ahora, pero usualmente es inconveniente. Típicamente, el análisis de memoria sugiere cuántas variables necesitamos y cuáles servicios necesitan esas variables. Cuando la memoria cambia, las variables correspondientes adquieren un nuevo valor, o dicho de otra forma, el estado de la declaración de la variable cambia y refleja el cambio de memoria en el tiempo. Por lo tanto, nos referiremos a las variables que implementen memoria como *variables de estado*.

Cada servicio en un programa corresponde a una función que puede usar funciones auxiliares. Un servicio que cambia la memoria de un programa es implementado con una función que usa la asignación (`=`) en alguna o algunas de las variables de estado. Para entender cómo una función debería modificar una variable de estado, necesitamos saber qué tipo de valores puede representar esta variable y cuál

es su propósito. En otras palabras, debemos definir un contrato y un propósito para las variables de estado de la misma manera en que desarrollamos contratos para definir una función.

Volvamos nuevamente al ejemplo de la libreta de direcciones. Ésta tiene una variable de estado: `libretaDeDirecciones`. Su propósito es representar una lista de registros, donde cada uno de éstos consiste en dos campos: un nombre y un número. Para documentar que `libretaDeDirecciones` puede representar sólo una lista de esta naturaleza, debemos agregar un contrato, tal como sigue:

```
1 import estructura
2 from lista import *
3
4 # registro: nombre(str) numero(int)
5 estructura.crear("registro", "nombre numero")
6
7 # libretaDeDirecciones: [registro]*
8 # para mantener los pares de nombres y numeros de telefono
9 libretaDeDirecciones = listaVacía
```

Por la definición del contrato, es posible usar `listaVacía` como el valor inicial de `libretaDeDirecciones`.

Del contrato de la variable de estado, podemos concluir que la siguiente asignación no tiene sentido:

```
1 libretaDeDirecciones = 5
```

La instrucción anterior asigna 5 a `libretaDeDirecciones`, lo cual no es una lista. Por lo tanto, la expresión viola el contrato. Pero la instrucción

```
1 libretaDeDirecciones = listaVacía
```

es válida, porque vuelve a asignar a `libretaDeDirecciones` su valor inicial. A continuación, otra asignación:

```
1 libretaDeDirecciones = lista(registro("Gonzalo", 42), \
2     libretaDeDirecciones)
```

Esto nos ayuda a entender cómo las funciones pueden cambiar el valor de `libretaDeDirecciones` de manera útil. Dado que `libretaDeDirecciones` consiste en una lista de registros, al ejecutar `lista(registro("Gonzalo", 42), libretaDeDirecciones)` se construye una lista más larga del tipo adecuado.

11.2.3 Funciones que inicializan memoria

Después de haber definido contratos y propósitos para las variables de estado de un programa, inmediatamente se define una función que asigna a la variable de estado un valor adecuado. Llamaremos a esta función como *función de inicialización* o *inicializador*. El inicializador de un programa es la primera función que es usada durante una ejecución; un programa también puede proveer de otros medios para invocar al inicializador.

Para nuestros ejemplos, los inicializadores son directos:

```

1 # inicLibretaDeDirecciones: None -> None
2 def inicLibretaDeDirecciones():
3     global libretaDeDirecciones
4     libretaDeDirecciones = listaVacía
5
6 # inicContador: None -> None
7 def inicContador():
8     global contador
9     contador = 0

```

A primera vista, pareciera que estos inicializadores no agregan mucho valor a nuestros programas: Ambos asignan a las respectivas variables de estado sus valores iniciales. Para ambos casos, sin embargo, el inicializador podría hacer más trabajo útil. En el primer ejemplo, se podría crear una interfaz gráfica para el usuario maneje la libreta de direcciones.

11.2.4 Funciones que cambian la memoria

Una vez que tenemos las variables de estado y sus inicializadores, nuestra atención se debe enfocar en diseñar las funciones que modifican la memoria de un programa. Estas funciones no sólo producen o consumen datos, sino que también afectan las definiciones de las variables de estado. Por lo tanto, hablaremos del *efecto* que estas funciones tienen sobre las variables de estado.

Revisemos las etapas más básicas de la receta de diseño y de cómo podemos acomodar los efectos en las variables de estado:

- **Análisis de Datos:**

Las funciones que afectan el estado de las variables pueden consumir y (posiblemente) producir datos. Por lo tanto, aún necesitamos analizar cómo representar la información y, si es necesario, introducir definiciones de estructuras y de datos.

En el ejemplo de la `libretaDeDirecciones`, la definición de los datos debe considerar el tipo de datos de los elementos de la lista y que la lista puede ser de un largo arbitrario.

- **Contrato, propósito y efecto:**

El primer cambio grande concierne al segundo paso. Además de especificar qué consume una función y qué produce, debemos describir adicionalmente a cuáles variables de estado afecta y cómo afecta a estas variables. El efecto de una función debe ser consistente con el propósito de la variable.

Por ejemplo, la especificación para la `libretaDeDirecciones` es la siguiente:

```

1 # agregarALibreta: str num -> None
2 # efecto: agregar registro(nombre, telefono) al comienzo de
3 # la libretaDeDirecciones
4 def agregarALibreta(nombre, telefono): ...

```

Podemos ver que la definición de `libretaDeDirecciones` se modifica de forma que es coherente con su propósito y su contrato.

- **Ejemplos del programa:**

Los ejemplos son tan importantes como siempre, pero formularlos puede ser más difícil. Como antes, debemos definir ejemplos que muestren la relación entre entradas y salidas, pero debido a

que las funciones ahora tienen efectos, necesitamos ejemplos que los ilustren.

Debido a que la variable de estado `libretaDeDirecciones` puede tener infinitos valores, es imposible hacer una lista comprehensiva de ejemplos. Lo importante es establecer algunos de ellos, dado que los ejemplos harán más fácil el desarrollar después el cuerpo de la función:

```

1 # si libretaDeDirecciones es lista vacia y
2 # evaluamos agregarALibreta("Maria", 1),
3 # entonces libretaDeDirecciones es lista(registro("Maria", 1),
4 # listaVacía)
5
6 # si libretaDeDirecciones es lista(registro("Claudio", 5),
7 # listaVacía) y evaluamos agregarALibreta("Gonzalo", 42),
8 # entonces libretaDeDirecciones es lista(registro("Gonzalo", 42),
9 # lista(registro("Claudio", 5), listaVacía))
10
11 # si libretaDeDirecciones es lista(registro1, lista(registro2,
12 # lista(..., listaVacía)...), y evaluamos
13 # agregarALibreta("Benjamin", 1729), entonces
14 # libretaDeDirecciones es lista(registro("Benjamin", 1729),
15 # lista(registro1, lista(registro2, lista(..., listaVacía))...))

```

El lenguaje de los ejemplos involucran palabras de naturaleza temporal. Esto no debería ser sorprendente, ya que las asignaciones enfatizan la noción de tiempo en la programación.

- **La Plantilla:**

La plantilla de una función que cambia el estado de las variables es igual que en una función ordinaria, pero el cuerpo debería contener expresiones de asignación (=) sobre variables de estado para especificar que las variables de estado están siendo modificadas. En Python se utiliza la instrucción `global` para identificar una variable de estado:

```

1 def funcionQueCambiaEstado(x, y, z):
2     global variableDeEstado
3     variableDeEstado = ...

```

En ocasiones, también pueden haber condiciones de por medio. Por ejemplo, se puede querer cambiar el estado de una variable dependiendo del valor de un parámetro o de otra variable de estado.

- **El cuerpo de la función:**

Como siempre, el desarrollo de la función completa requiere de un entendimiento sólido de los ejemplos (cómo son calculados) y de la plantilla. Para las funciones con efectos, la expresión de asignación es el paso más demandante. En algunos casos, el lado derecho de la asignación involucra sólo operaciones primitivas, los parámetros de la función y la variable de estado (o varias de ellas). En otros casos, lo mejor es escribir una función auxiliar (sin efecto) que consuma el valor actual de la variable de estado y los parámetros de la función, y que produzca un nuevo valor para la variable de estado.

La función `agregarALibreta` es un ejemplo del primer tipo. El lado derecho de la asignación consiste en la variable `libretaDeDirecciones`, `lista` y la creación de un `registro`.

- **Testing:**

Es una tarea complicada verificar que las funciones en estos casos tengan el efecto deseado.

Hay dos maneras de testear funciones con efectos. Primero, podemos asignar la variable de estado a un estado deseado, aplicar la función, y luego verificar que la función tiene el efecto deseado y el resultado esperado.

Para el ejemplo del contador es bastante sencillo:

```
1 contador = 3
2 siguiente = cuenta()
3 assert siguiente == 4
```

El caso de la libreta de direcciones es más complicado:

```
1 libretaDeDirecciones = listaVacía
2 agregarALibreta("Benjamin", 1729)
3 assert lista(registro("Benjamin", 1729), listaVacía) \
4     == libretaDeDirecciones
```

En este test sólo verificamos que "Benjamin" y 1729 hayan sido agregados a una lista vacía.

En la segunda forma de testear, podemos capturar el estado de una variable de estado antes de que sea testada, luego aplicar la función que cambia la memoria y finalmente conducir a los tests apropiados. Considere la siguiente expresión:

```
1 libretaActual = libretaDeDirecciones
2 agregarALibreta("John", 10)
3 assert lista(registro("John", 10), libretaActual) \
4     == libretaDeDirecciones
```

Ésta define en `libretaActual` el valor de `libretaDeDirecciones` al comienzo de la evaluación, y al final verifica que la entrada o registro ha sido agregada al frente de la lista y que nada ha cambiado para el resto de la libreta.

Para conducir a pruebas sobre funciones con efectos, especialmente a pruebas del segundo tipo, es útil de abstraer la expresión del test en una función:

```
1 # testLibretaDeDirecciones: str num -> bool
2 # para determinar si agregarALibreta tiene el efecto deseado
3 # sobre libretaDeDirecciones y no mas que eso
4 # efecto: el mismo que agregarALibreta(nombre, numero)
5 def testLibretaDeDirecciones(nombre, numero):
6     libretaActual = libretaDeDirecciones
7     agregarALibreta(nombre, numero)
8     return lista(registro(nombre, numero), libretaActual) \
9         == libretaDeDirecciones
```


Usando esta función, ahora podemos fácilmente testear `agregarALibreta` varias veces y asegurarnos que cada vez el efecto es el adecuado:

```
1 assert testLibretaDeDirecciones("Claudio", 5)
2 assert testLibretaDeDirecciones("Maria", 1)
3 assert testLibretaDeDirecciones("Gonzalo", 42)
4 assert testLibretaDeDirecciones("John", 10)
```

- **Reuso futuro:**

Una vez que tenemos un programa completo y testeado, deberíamos recordar *qué* calcula y cuáles son sus *efectos*. Sin embargo, no necesitamos saber cómo funciona. Si nos encontramos con una situación donde se necesita el mismo cálculo y los mismos efectos, podemos rehusar el programa como si fuera una operación primitiva. En presencia de efectos, es mucho más difícil reusar una función que en el mundo de los programas algebraicos o sin efectos, es decir, utilizando sólo programación funcional.

```
1 import estructura
2 from lista import *
3
4 # registro: nombre(str) numero(int)
5 estructura.crear("registro", "nombre numero")
6
7 # Variables de estado:
8 # libretaDeDirecciones: [registro]*
9 # para mantener los pares de nombres y numeros de telefono
10 libretaDeDirecciones = listaVacia
11
12 # agregarALibreta: str num -> None
13 # proposito: la funcion siempre produce None
14 # efecto: agregar registro(nombre, numero) al principio de
15 # libretaDeDirecciones
16
17 # Encabezado:
18 # def agregarALibreta(nombre, numero): ...
19
20 # Ejemplos:
21 # si libretaDeDirecciones es lista vacia y
22 # evaluamos agregarALibreta("Maria", 1),
23 # entonces libretaDeDirecciones es
24 # lista(registro("Maria", 1), listaVacia)
25
26 # si libretaDeDirecciones es lista(registro("Claudio", 5), \
27 # listaVacia) y evaluamos agregarALibreta("Gonzalo", 42),
28 # entonces libretaDeDirecciones es lista(registro("Gonzalo", 42),
29 # lista(registro("Claudio", 5), listaVacia))
30
31 # Plantilla: omitida
32
33 # Definicion:
34 def agregarALibreta(nombre, telefono):
```

```

35 global libretaDeDirecciones
36 libretaDeDirecciones = lista(registro(nombre, telefono), \
37     libretaDeDirecciones)
38
39 # Tests:
40 # testLibretaDeDirecciones: str num -> bool
41 # para determinar si agregarALibreta tiene el efecto deseado
42 # sobre libretaDeDirecciones y no mas que eso
43 # efecto: el mismo que agregarALibreta(nombre, numero)
44 def testLibretaDeDirecciones(nombre, numero):
45     libretaActual = libretaDeDirecciones
46     agregarALibreta(nombre, numero)
47     return lista(registro(nombre, numero), libretaActual) \
48         == libretaDeDirecciones
49
50 assert testLibretaDeDirecciones("Claudio", 5)
51 assert testLibretaDeDirecciones("Maria", 1)
52 assert testLibretaDeDirecciones("Gonzalo", 42)
53 assert testLibretaDeDirecciones("John", 10)

```

11.3 Estructuras mutables

Consideremos las estructuras que se pueden definir con el módulo `estructura`:

```

1 >>> import estructura
2 >>> estructura.crear("artista", "nombre instrumento")
3 >>> p = artista("Michael Weikath", "guitar")
4 >>> p.instrumento
5 "guitar"
6 >>> p.instrumento = "bass"

```

```

Traceback (most recent call last):
File "<stdin>", line 1, in <module>
AttributeError: can't set attribute

```

El error significa que nuestras estructuras no son *mutables*, esto es, sus campos no son variables de estado. Para manejar estructuras mutables, el módulo `estructura` provee la función `mutable`:

```

1 >>> import estructura
2 >>> estructura.mutable("artista", "nombre instrumento")
3 >>> p = artista("Michael Weikath", "guitar")
4 >>> p.instrumento
5 "guitar"
6 >>> p.instrumento = "bass"
7 >>> p.instrumento
8 "bass"

```

La modificación de la variable de estado de la estructura modifica el valor sólo en la definición de `p`, no la estructura completa.

Estudiemos en mayor profundidad el comportamiento “inusual” al realizar asignaciones en estructuras mutables. Aquí hay un primer ejemplo:

```
1 >>> estructura.mutable("artista", "nombre instrumento")
2 >>> p = artista("Michael Weikath", "guitar")
3 >>> q = p
4 >>> p.instrumento = "bass"
5 >>> q.instrumento
6 "bass"
```

Este ejemplo difiere del primero en dos formas. Primero, define a `q` como `p`. Segundo, la última expresión se hace sobre `q` y no sobre `p`.

Lo que acabamos de ver es el efecto de *compartir* el efecto de una asignación, lo que significa que una modificación de una estructura afecta al programa en más de un lugar. El efecto de compartir también es visible dentro de listas, como muestra el siguiente ejemplo:

```
1 >>> estructura.crear("lista", "val sig")
2 >>> estructura.mutable("artista", "nombre instrumento")
3 >>> unaLista = lista(artista("Michael Weikath", "guitar"), None)
4 >>> (unaLista.val).artista = "Roland Grapow"
5 >>> (unaLista.val).artista
6 "Roland Grapow"
```

Finalmente, los efectos pueden ser compartidos entre los elementos de diferente listas:

```
1 >>> estructura.crear("lista", "val sig")
2 >>> estructura.mutable("artista", "nombre instrumento")
3 >>> q = lista(artista("Michael", "guitar"), None)
4 >>> r = lista(q.val, lista((q.val).instrumento, None))
5 >>> (q.val).instrumento = 'vocals'
6 >>> (r.val).instrumento
7 "vocals"
```

La nueva definición introduce la variable `r`, una lista con dos elementos. Como `r` contiene al `artista` como primer elemento y como el campo `instrumento` del `artista` es `"vocals"`, el resultado es `"vocals"`. Sin embargo, el programa todavía tiene conocimiento de `"guitar"` en alguna parte. ¿Puede encontrar dónde?

En resumen, la asignación sobre estructuras mutables nos da bastante poder. No sólo podemos crear nuevas estructuras y revelar sus contenidos, sino que también podemos *cambiar* sus contenidos, mientras que las estructuras se mantienen igual. Ahora tenemos que comprender qué significa esto para el diseño de programas.

11.4 Diseñar funciones que modifican estructuras

En la sección anterior se introdujo la idea de estructuras mutables. Ahora necesitamos saber cuándo y cómo aplicar este nuevo poder.

11.4.1 ¿Por qué mutar estructuras?

Cada vez que aplicamos el constructor de estructuras, estamos creando una nueva estructura. En algunas ocasiones, esto es justo lo que queremos. Considere una función que consume una lista de registros de personal y produce una lista de registros telefónicos. Los registros de personal pueden contener información como la dirección de cada persona, el número de teléfono, la fecha de nacimiento, salario, etc. Un registro telefónico debería contener el nombre y el número de teléfono y nada más. Este tipo de programas definitivamente debería crear una nueva estructura por cada valor en la lista dada.

Sin embargo, en otras ocasiones crear una nueva estructura no corresponde a la intuición. Suponga que queremos darle un aumento a alguien. La única forma de lograr esto hasta el momento era crear un nuevo registro de personal que contuviera toda la información anterior y la nueva información del salario. O, suponga que alguien se cambió de casa y tiene un nuevo número de teléfono, y nos gustaría actualizar nuestra lista de contactos. Tal como el programa que cambia el nivel de salario de una persona, el programa que actualiza la lista de contactos crearía un nuevo registro. En la realidad, sin embargo, no crearíamos un nuevo registro de personal ni una nueva entrada en la lista de contactos. En vez de eso, corregiríamos el registro actual en ambos casos. Un programa debería ser capaz de hacer la misma acción correctiva, y con mutación, podemos desarrollar tales programas.

11.4.2 Receta de diseño estructural y con mutación

No necesitamos una nueva receta si consideramos la mutación dentro de nuestra programación, siempre y cuando los campos mutados contengan valores atómicos (por ejemplo, no consisten en otras estructuras). Mientras el diseño de programas sin mutación requiere la combinación de valores, programar con mutación requiere la combinación de efectos, por lo que la clave es agregar una descripción bien formulada del efecto de una función en su contrato, y declarar ejemplos que ilustren tales efectos.

Suponga que nos dan una estructura y una definición de datos para registros de personal:

```
1 import estructura
2
3 # Un registro de empleado (RE) es una estructura
4 # RE(n, d, s)
5 # donde n es un string, d es un string y s es un entero
6
7 # RE: nombre(str) direccion(str) salario(int)
8 estructura.mutable("RE", "nombre direccion salario")
```

Una función que consume uno de estos registros se basa en la siguiente plantilla:

```
1 def funcionParaEmpleado(re):
2     ...re.nombre...
3     ...re.direccion...
4     ...re.salario...
```

Considere una función para subir el salario a un empleado:

```
1 # aumentarSalario: RE int -> None
2 # efecto: modificar el campo salario de unEmpleado al sumar unAumento
3 def aumentarSalario(unEmpleado, unAumento):
4     ...
```

El contrato especifica que la función recibe un RE y un número entero. El propósito es también un efecto, que explica cómo el argumento de `aumentarSalario` se modifica.

Desarrollar ejemplos para `aumentarSalario` requiere las técnicas vistas anteriormente. Específicamente, debemos poder comparar el estado anterior y posterior de una estructura RE:

```
1 re1 = empleado("John", "Su casa", 1729)
2 aumentarSalario(re1, 3000)
3 assert re1.salarario == 4729
```

Ahora podemos usar la plantilla y el ejemplo para definir la función:

```
1 # aumentarSalario: RE int -> None
2 # efecto: modificar el campo salario de unEmpleado al sumar unAumento
3 def aumentarSalario(unEmpleado, unAumento):
4     unEmpleado.salarario = unEmpleado.salarario + unAumento
```

Como es usual, la definición completa de la función usa sólo una de las subexpresiones disponibles en la plantilla, pero ésta nos recuerda qué información podemos usar: los argumentos y sus partes, y qué partes podemos modificar.

¿Qué pasa si los campos de una estructura son, a su vez, estructuras? Suponga que queremos simular un juego de cartas. Cada carta tiene dos características importantes: su *tipo* y su *valor*. La colección de cartas de un jugador se llama *mano*. Vamos a suponer que la mano de un jugador nunca está vacía, esto es, siempre tiene al menos una carta en su mano.

Una mano consiste de una estructura `mano` con `valor`, `tipo` y `siguiente` como campos. El campo `siguiente` puede contener dos tipos de valores: vacío (`None`), que indica que no hay más cartas, y una estructura `mano`, que contiene las cartas restantes. De una perspectiva más general, una `mano` es una lista de cartas, pero sólo la última contiene vacío (`None`) como valor en `siguiente`.

Al principio, un jugador no tiene cartas. Al sacar la primera se creará su mano. Las otras cartas son puestas en la mano existente cuando sean necesarias. Esto llama a dos funciones: una para crear la mano y otra para insertar una carta en la mano. Debido a que la mano existe sólo una vez y corresponde a un objeto físico, es natural pensar que la segunda función es la que modifica un valor existente en vez de crear uno nuevo. Por ahora vamos a aceptar esta premisa y explorar sus consecuencias.

Crear una mano es simple y fácil de implementar:

```
1 # crearMano: valor tipo -> mano
2 # para crear una mano de una sola carta de v y t
3 def crearMano(v, t):
4     return mano(v, t, None)
```

Agregar una carta a la mano es un poco más difícil. Para simplificar, diremos que un jugador siempre agrega nuevas cartas al final de la mano. En este caso debemos procesar un valor arbitrariamente grande, lo que significa que necesitamos una función recursiva:

```
1 # agregarAlFinal: valor tipo mano -> None
2 # efecto: agrega la carta con valor v y tipo t al final de unaMano
3 def agregarAlFinal(v, t, unaMano):
4     ...
```

Esta especificación dice que la función tiene un valor invisible como resultado que se comunica con el resto del programa sólo por medio de sus efectos.

Veamos algunos ejemplos:

```
1 mano0 = crearMano(13, "trebol")
```

Si fuéramos a evaluar la siguiente expresión:

```
1 agregarAlFinal(1, "diamante", mano0)
```

en el contexto de esta definición, `mano0` se convierte en una mano con dos cartas: un káiser de trébol seguido de un as de diamante. Si agregamos:

```
1 agregarAlFinal(2, "corazones", mano0)
```

En este contexto, `mano0` es una mano con tres cartas. La última es un 2 de corazones. En términos de una evaluación, la definición de `mano0` debería cambiar a:

```
1 mano0 = crearMano(13, "trebol", crearMano(1, "diamante", \
2     crearMano(2, "corazones", None)))
```

Dado que el `valor` y el `tipo` pasados a `agregarAlFinal` son valores atómicos, la plantilla debe estar basada en la definición de los datos de tipo `mano`:

```
1 def agregarAlFinal(valor, tipo, unaMano):
2     if unaMano.siguiente == None:
3         ... unaMano.valor ... unaMano.tipo ...
4     else:
5         ... unaMano.valor ... unaMano.tipo ...
6         agregarAlFinal(valor, tipo, unaMano.siguiente)
```

La plantilla consiste en dos cláusulas, que verifican el contenido del campo `siguiente` de `unaMano`. Es recursiva en la segunda cláusula, debido a que la definición de datos de las `manos` es auto-referencial en esa cláusula. En resumen, la plantilla es convencional.

El siguiente paso a considerar es cómo la función debería afectar a `unaMano` en cada cláusula:

1. En el primer caso, el campo `siguiente` de `unaMano` es vacío (`None`). En ese caso, podemos modificar el campo `siguiente` de forma que contenga la nueva carta:
`unaMano.siguiente = crearMano(v, t)`
Recuerde que la nueva `mano` creada contiene vacío (`None`) en su campo `siguiente`.
2. En el segundo caso, la recursión natural agrega una nueva carta la final de `unaMano`. De hecho, debido a que la `unaMano` dada no es la última en la cadena, la recursión natural resuelve el problema.

Esta es la definición completa de `agregarAlFinal`:

```
1 # agregarAlFinal: valor tipo mano -> None
2 # efecto: agrega la carta con valor v y tipo t al final de unaMano
```

```
3 def agregarAlFinal(valor, tipo, unaMano):  
4     if unaMano.siguiente == None:  
5         unaMano.siguiente = crearMano(valor, tipo)  
6     else:  
7         agregarAlFinal(valor, tipo, unaMano.siguiente)
```

Capítulo 12

Estructuras Indexadas

En este capítulo se estudiarán estructuras de datos indexadas, que permiten acceder a los valores de los datos almacenados en ellas a través de un índice. Algunas de estas estructuras son mutables, pero otras no. También se estudiarán instrucciones que permiten iterar sobre los valores de estas estructuras indexadas.

12.1 Arreglos

Un *arreglo* es una estructura de datos mutable que consiste en una secuencia contigua de un número fijo de elementos homogéneos almacenados en la memoria. En la siguiente figura se muestra un arreglo de enteros con diez valores:

<i>índice</i>	0	1	2	3	4	5	6	7	8	9
<i>elementos</i>	80	45	2	21	92	17	5	65	14	34

Figura 12.1: Ejemplo de un arreglo con diez valores.

Para acceder a un elemento del arreglo se utiliza un índice que identifica a cada elemento de manera única. Los índices son números enteros correlativos y, en la mayoría de los lenguajes de programación, comienzan desde cero. Por lo tanto, si el arreglo contiene n elementos el índice del último elemento del arreglo es $n - 1$. Una ventaja que tienen los arreglos es que el costo de acceso de un elemento del arreglo es constante, es decir no hay diferencias de costo entre acceder el primer, el último o cualquier elemento del arreglo, lo cual es muy eficiente. La desventaja es que es necesario definir a priori el tamaño del arreglo, lo cual puede generar mucha pérdida de espacio en memoria si se definen arreglos muy grandes para contener conjuntos pequeños de elementos.

Python posee una biblioteca para crear y manipular arreglos de variables numéricas, pero en vez de esta estructura estudiaremos otra que denominaremos *listas de Python*. Estas listas son mucho más flexibles que los arreglos, permiten guardar cualquier tipo de dato dentro de éstas (como las listas recursivas que ya conocen) y además proveen de varias funciones útiles.

12.2 Listas de Python

Las listas de Python son estructuras mutables definidas en el lenguaje Python que, al igual que los arreglos, indexan los elementos que se insertan en ella. Los índices son números enteros correlativos y parten en cero. Las siguientes instrucciones sirven para crear una lista vacía:

```
1 >>> unaLista = list() # lista vacía
2 >>> otraLista = [] # lista vacía
```

También se pueden crear listas con valores ya insertados, como se muestra en el ejemplo siguiente. Recuerde que las listas de Python permiten insertar datos de cualquier tipo en sus casilleros:

```
1 >>> Enteros = [1, 2, 3, 4, 5] # lista de int
2 >>> Strings = ['casa', 'arbol', 'planta', 'auto'] # lista de str
3 >>> Todo = ['a', 17, True, 9.5] # lista con datos de distinto tipo
```

Dos listas se pueden concatenar con el operador '+', o se puede repetir varias veces el contenido de una lista multiplicándola por un escalar:

```
1 >>> lista1 = [10, 20]
2 >>> lista2 = [50, 60]
3 >>> lista2 + lista1
4 [50, 60, 10, 20]
5 >>> lista1 * 3
6 [10, 20, 10, 20, 10, 20]
7 >>> lista1 + [30]
8 [10, 20, 30]
```

Para conocer el largo de una lista (esto es, cuántos valores contiene), se utiliza la función `len` de Python:

```
1 >>> lista = ['ana', 'maria', 'luisa', 'teresa']
2 >>> len(lista)
3 4
```

Para acceder a los valores en una lista se utiliza el índice del casillero correspondiente. No olvide que el primer valor de la lista corresponde al índice cero. Por ejemplo:

```
1 >>> lista = ['ana', 'maria', 'luisa', 'teresa']
2 >>> lista[0]
3 'ana'
4 >>> lista[3]
5 'teresa'
6 >>> lista[4]
```

```
Traceback (most recent call last):
File "<pyshell#20>", line 1, in <module>
lista[4]
IndexError: list index out of range
```

Note en el código anterior que cuando se intentó acceder a un casillero de la lista con un valor de índice inválido (por ejemplo, se sale del rango de los índices de la lista), se produjo un `IndexError`.

Las listas son estructuras mutables, por lo que se pueden modificar los valores almacenados en sus casilleros:

```
1 >>> lista = [10, 20, 30, 40, 50, 60]
2 >>> lista[3] = 'X'
3 >>> lista
4 [10, 20, 30, 'X', 50, 60]
```

Una función útil predefinida de Python es la función `range`, que retorna una lista de Python (`list`) que contiene una progresión aritmética de enteros. Esta función provee tres contratos distintos:

```
1 range(stop): int -> list(int)
2 range(start, stop): int int -> list(int)
3 range(start, stop, step): int int int -> list(int)
```

La primera versión retorna una lista que comienza en cero y termina en `stop - 1`. La segunda versión retorna la lista `[start, start+1, ..., stop-2, stop-1]`. En la tercera versión, el parámetro `step` especifica el incremento (o decremento) entre números consecutivos de la lista.

Note que para una lista de largo n , la función `range(n)` retorna la lista `[0, 1, 2, ..., n-1]`, que corresponden a los índices válidos para acceder a los casilleros de la lista.

12.3 Iterar sobre estructuras indexadas

Se define un ciclo como un bloque de instrucciones que se repiten de acuerdo a ciertas condiciones definidas. El ejecutar las instrucciones de un ciclo una vez se denomina iteración. Es necesario entonces contar con instrucciones que definan las condiciones que harán que se itere un ciclo o que se terminen las iteraciones. Para este fin, existen dos instrucciones principales: `for` y `while`.

12.3.1 Instrucción `for`

La instrucción `for` de Python permite iterar sobre una estructura indexada. La sintaxis de la instrucción `for` es la siguiente:

```
1 for identificador in estructura:
2     # bloque de instrucciones
```

donde `identificador` es una variable y `estructura` es el identificador de la estructura sobre la cual se va a iterar. En cada iteración del ciclo, se le asignará a la variable un valor almacenado en la estructura indexada (por ejemplo, los valores almacenados en una lista), que se indica después de la instrucción `in`. Por ejemplo:

```
1 >>> lista = [10, 20, 30, 40, 50]
2 >>> for valor in lista:
3     print valor
4 10
5 20
6 30
7 40
8 50
```

En el ejemplo anterior, la instrucción del ciclo se ejecuta cinco veces, una vez por cada asignación realizada a la variable `valor`.

Note que es posible obtener el mismo resultado anterior si es que a la variable se le asigna un valor de una lista retornada por `range`, y luego dicho valor se ocupa como índice para acceder a los distintos casilleros de la lista:

```
1 >>> lista = [10, 20, 30, 40, 50]
2 >>> for indice in range(len(lista)):
3     print lista[indice]
4 10
5 20
6 30
7 40
8 50
```

Para acceder sólo a un cierto rango de la lista, se utiliza la notación `[i:j]`, donde `i` indica el primer índice a ser considerado, y `j-1` indica el último índice a ser considerado (note que el índice `j` se excluye del rango). Por ejemplo:

```
1 >>> cuadrados = [0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
2 >>> # alternativamente: cuadrados = [x**2 for x in range(0,11)]
3 >>> cuadrados[3:6] # solo mostrar indices 3, 4 y 5
4 [9, 16, 25]
```

Finalmente, note que una lista puede contener a otra lista en sus casilleros. Con esto es posible implementar una matriz (una lista de listas de tamaño fijo). Para acceder a los valores de la matriz, se pueden escribir directamente los valores de los índices de la fila y la columna del valor requerido:

```
1 >>> matriz = [[10, 20, 30], [40, 50, 60], [70, 80, 90]]
2 >>> matriz[1][2]
3 60
```

12.3.2 Instrucción `while`

La instrucción `while` es más general que la instrucción `for` de Python, y permite iterar sobre un bloque de instrucciones. La sintaxis de la instrucción `while` es la siguiente:

```
1 while condicion: # condicion es de tipo boolean
2     # bloque de instrucciones
```

Las instrucciones del ciclo `while` se ejecutarán mientras `condicion` (que corresponde a cualquier expresión condicional) sea `True`. En particular, si se coloca como condición el valor de verdad `True`, el ciclo se repetirá indefinidamente, lo que se denomina un *loop infinito*. Es responsabilidad del programador asegurarse que, salvo que por diseño se requiera programar un loop infinito, la condición es `False` en algún momento, lo que permite al programar salir del ciclo y seguir ejecutando el resto de sus instrucciones.

La instrucción `while` se puede utilizar para iterar sobre los valores de una lista de la siguiente forma:

```
1 >>> lista = [10, 20, 30, 40, 50]
2 >>> indice = 0
3 >>> while indice < len(lista):
4     print lista[indice]
5     indice = indice + 1
6 10
7 20
8 30
9 40
10 50
```

12.4 Strings como secuencias indexadas de caracteres

En Python, los strings también corresponden a una estructura indexada (pero inmutable, al contrario de las listas). En efecto, es posible utilizar la instrucción `for` para iterar sobre todas las letras de un string:

```
1 >>> palabra = 'algoritmo'
2 >>> for caracter in palabra:
3     print caracter
4 a
5 l
6 g
7 o
8 r
9 i
10 t
11 m
12 o
```

12.5 Diccionarios de Python

Un diccionario de Python es una estructura indexada mutable en donde los índices ya no son números correlativos (como en el caso de las listas de Python), sino que son *llaves* que corresponden a cualquier tipo de dato inmutable. Por ejemplo, los Strings y los números pueden ser llaves en un diccionario, pero una lista (que es mutable) no lo puede ser.

Un diccionario guarda pares *llave:valor* en un orden arbitrario. Las llaves deben ser únicas, de forma de poder identificar cada valor almacenado en el diccionario de forma unívoca. Para crear diccionarios, se utiliza la siguiente sintaxis:

```
1 >>> diccionario1 = {} # diccionario vacio
2 >>> diccionario2 = dict() # otro diccionario vacio
3 >>> telefonos = {'ana':21234567, 'pedro':29876543, 'sofia':23344556}
```

Para recuperar un valor se utiliza la misma notación que en las listas, pero colocando la llave correspondiente:

```
1 >>> telefonos['pedro']  
2 29876543  
3 >>> telefonos['ana']  
4 21234567
```

Capítulo 13

Depuración¹

13.1 ¿Qué es la depuración?

En un mundo perfecto, nuestros programas funcionan la primera vez que lo ejecutamos. Dada su experiencia, ya saben que, desafortunadamente, no vivimos en un mundo perfecto.

Depurar es el proceso de entender por qué un programa no está funcionando. Después de darse cuenta que el programa no funciona (con el testing), queremos entender por qué el programa no funciona como esperábamos. Después de entender la fuente del error, viene el paso de arreglar el error, pero esto es usualmente mas fácil que entender por qué el programa falló.

Testear y depurar son cosas muy diferentes. Cuando testeamos, estamos comparando la entrada y la salida de un programa con su especificación. Cuando depuramos, estamos estudiando los eventos que hicieron surgir un error.

Hay dos tipos principales de testing: el test unitario y el test de integración. El test unitario prueba una pieza simple de un programa, idealmente aislada del resto del programa. Éste es el tipo de testing que hemos estudiado hasta ahora. El otro tipo de testing es el test de integración, que prueba un programa grande completo. Al probar un programa completo pueden surgir nuevos errores, incluso cuando hemos testado bien cada función en el programa individualmente. Esto es porque al unir las funciones en un programa grande, es mas probable que encontremos casos excepcionales que no hemos incluido en nuestros tests. Cuando esto ocurre, tenemos que depurar nuestro programa.

Depurar es una habilidad que se aprende, nadie lo hace bien la primera vez, pero es una de las características que diferencia un buen programador de otros. Usualmente, aprender a depurar es un proceso lento, y toma tiempo hasta que uno puede hacer mucho progreso muy rápido. Un aspecto positivo de la depuración es que no sólo se aplica en programación, sino que esta habilidad se puede transferir a otras áreas. Depurar es un proceso de ingeniería que se puede ocupar en muchos sistemas complejos, como por ejemplo en experimentos de laboratorio.

Existen dos herramientas básicas para hacer depuración: la instrucción `print` y la lectura de código. Ser capaz de entender qué hace exactamente el código (no lo que uno piensa que hace) es probablemente la habilidad más importante para hacer depuración. Por esto, la instrucción `print` puede ser útil para conocer, por ejemplo, el valor actual de una variable de estado, y si ésta se modifica correctamente después de alguna instrucción o invocación a función. Una alternativa a la instrucción

¹Parte de este capítulo fue traducido al español y adaptado de: Eric Grimson and John Guttag, 6.00 Introduction to Computer Science and Programming, Fall 2008. (Massachusetts Institute of Technology: MIT OpenCourseWare). <http://ocw.mit.edu> (accessed 11 04, 2011). License: Creative Commons Attribution-Noncommercial-Share Alike.

`print` es la instrucción `assert`, que nos permite parar el programa cuando algo inesperado ocurre. Hay herramientas más avanzadas para realizar este tipo de procedimiento de depuración, llamadas *depuradores* (en inglés *debuggers*); IDLE tiene uno. Estas herramientas son un poco más complejas, así que no vamos a verlas, pero a los interesados les recomendamos encarecidamente que las investiguen.

Lo más importante que uno tiene que recordar cuando depura es ser sistemático. Esto es lo que separa buenos depuradores de malos depuradores: los buenos depuradores han encontrado una manera sistemática de buscar errores en programas. Lo que hacen es reducir el espacio de búsqueda en el programa en donde puede estar el error, hasta encontrar la fuente de la error. Visto de esta manera, depurar es un proceso de búsqueda en el código de un programa. De la misma manera que cuando una busca un valor en una lista, uno no toma elementos al azar, sino que hace un recorrido sistemático de la lista. Desafortunadamente, esta búsqueda al azar es lo que hace mucha gente cuando están buscando errores.

13.2 El proceso de depuración

Depurar empieza cuando nos damos cuenta que hay un error en el programa. Lo primero que se debe hacer es estudiar el texto del programa, preguntándose: ¿Como el programa podría haber dado este resultado? Es importante notar que la pregunta no es “¿Por qué el programa no ha dado el resultado esperado?”, sino que “¿Por qué ha producido este resultado?” Formular la pregunta de esta manera permite encontrar más fácilmente el error, porque la pregunta es más cercana a lo que hace el programa.

La segunda pregunta es: ¿Será este error parte de una familia de errores? La idea de esta pregunta es que uno no quiere arreglar un defecto nada más, sino que quiere asegurarse que el programa está libre de errores. Si un error proviene, por ejemplo, de un manejo equivocado de la mutación en listas, conviene verificar todas las ocurrencias de manejo de listas en el programa, para ver si el mismo error está presente en otros contextos. Más que arreglar sólo un defecto, uno tiene que detenerse y verificar si el defecto ocurre más de una vez, arreglando todas las instancias del error de una vez, y no empezar la búsqueda de cero cuando un error parecido aparece más adelante. Esto también es parte de ser sistemático en la búsqueda de errores.

La última pregunta es ¿cómo localizar y corregir el error? Para esto se usa el método científico. El método científico empieza estudiando los datos disponibles. En este caso, los datos disponibles son los resultados de los tests del programa. Estos son los resultados equivocados, pero también los resultados correctos. Esto es porque el programa puede funcionar en algunos casos y en otros no; a veces, entender por qué el programa funcionó en un caso y no en el otro permite entender la causa del defecto, basado en la diferencia entre el caso correcto y el caso equivocado.

La otra gran pieza de información es el texto del programa. Cuando estudien el programa, recuerden que no lo entienden, porque si lo hubieran entendido de verdad el programa no tendría un defecto. Entonces, hay que leer el programa con un ojo crítico.

Mientras uno estudia los datos, el método científico nos hace formular una hipótesis consistente con los datos. No sólo una parte de los datos; todos los datos. Basado en esta hipótesis, se diseña un experimento repetible.

13.3 Depurar programas con el método científico

Les recuerdo que un experimento científico tiene el potencial de demostrar que la hipótesis es falsa. Si no es posible hacer esto, el experimento no es válido científicamente. Para esto, también tenemos

que saber cuál es el resultado esperado. Uno tiene típicamente una hipótesis, y la hipótesis dice que el resultado a obtener tiene que ser un resultado X. Si el resultado no es X, se comprobó que la hipótesis es falsa. Este es el punto donde mucha gente falla en la depuración: no se toman el tiempo de pensar qué es lo que tendría que devolver el programa, o sea, de formar una hipótesis. Si éste es el caso, no están siendo sistemáticos al interpretar los resultados del experimento. Antes de ejecutar cualquier test, tienen que saber que esperan que realice el programa.

Un problema potencial es el problema de la reproducibilidad de los resultados. En general tratamos de tener programas que hacen la misma cosa cada vez que se ejecutan, porque es mas fácil de depurarlos de esta manera. Sin embargo, esto no es siempre posible. Algunos programas hacen uso de números aleatorios, lo que impide que cada ejecución del programa sea idéntica. Dado que para depurar uno tiene que tratar de hacer la ejecución reproducible, algo que se puede hacer en este caso es definir la semilla del algoritmo de generación de números aleatorios. Un algoritmo de generación de números aleatorios genera una secuencia compleja, pero determinística, de números, y si lo inicializamos en un mismo estado (la semilla), siempre reproduce la misma secuencia de números.

Otra cosa que impide la reproducibilidad son las interacciones con el usuario. Si un programa reacciona a interacciones del usuario, no va a ser reproducible. En este caso, hay que hacer que estas interacciones sean definidas en una pequeña parte del programa, de tal manera que la mayor parte del programa sea testeable y depurable de una manera reproducible.

Si pensamos en cómo diseñar un experimento de depuración de un programa, hay varios objetivos:

- El primero es de encontrar la entrada más pequeña posible que conduzca al error. En muchos casos, un programa se va a ejecutar por mucho tiempo antes de encontrar un error. Esto es poco práctico, así que hay que tratar de reducir el problema para encontrarlo de manera más rápida. Por ejemplo, si un juego de palabras no funciona con palabras de 12 letras, podemos probar como funciona con palabras de 3 letras. Si la misma falla ocurre con 3 letras en vez de 12, esto simplifica mucho la resolución del problema, ya que el espacio de búsqueda se reduce bastante. El proceso usual es de tratar de reducir la entrada paso a paso: primero con 11 letras, despues 10, etc., hasta que el programa empieza a funcionar de nuevo.
- El segundo objetivo es encontrar la parte del programa que es la más probable de ser "culpable". En ambos casos, es recomendable hacer una búsqueda binaria. La idea es que tratamos de descartar la mitad de los datos, o la mitad del programa a cada paso. Esto permite encontrar el lugar del problema rápidamente, incluso si el programa o los datos son grandes. En el caso de un programa, la idea es de ocupar la instrucción `print` para imprimir valores intermedios del programa, y determinar si están correctos. La búsqueda binaria en este caso nos va a hacer empezar en la mitad del programa. Si no encontramos el error, esto quiere decir que se ubica en la segunda mitad del programa. Si lo encontramos, quiere decir que está ubicado en la primera mitad. Después seguimos buscando en cuartos del programa, y asi sucesivamente, hasta llegar a la línea que contiene el error. Con este proceso, a cada paso descartamos la mitad del programa como "inocente" del error.

En resumen, depurar programas es una tarea compleja que requiere ser sistemático y entender bien qué es lo que hace el programa. De esta forma, será posible encontrar el error para corregirlo. Programar es una tarea compleja y suceptible a muchos errores, incluso los mejores programadores cometen errores al escribir programas, por lo que es muy importante para todo programador el practicar la habilidad de depuración.